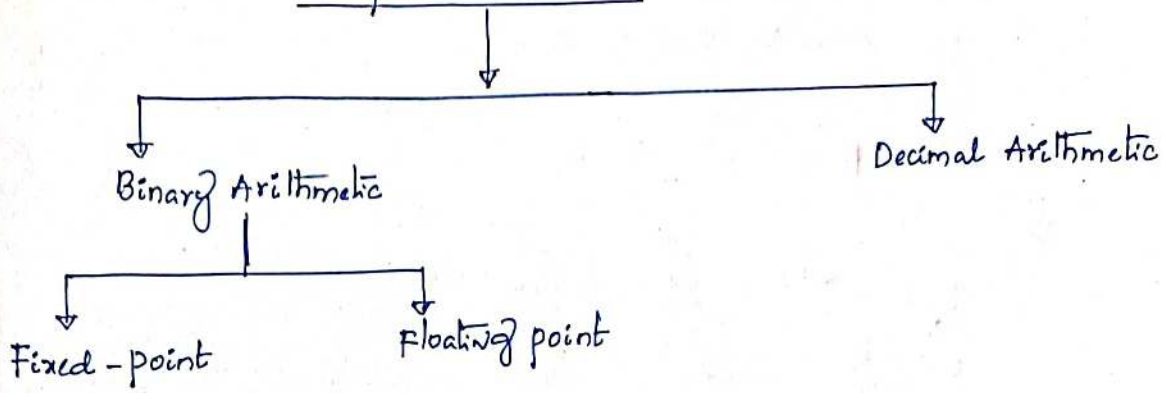
## Computer Arithmetic




* **Fixed-point Arithmetic**: Are those which have the radix point in a specific position of a computer word.

* **Floating-point Arithmetic**: Is an automatic way to keep track of the radix -point

**Note**

1101.0101
↑ Radix point

* **Sign Extension**:

If bit number (n) is smaller than the size of Register or memory Location, then sign extension rule has to be followed.

**Rule**:

- If the number is positive (S=0), the vacant MSB's are filled with 0's.
- If the number is negative (S=1), all the vacant MSB's are filled with 1's.

The above concept is called Sign extension.

P) Show how the following 5 bit signed numbers are stored in 8-bit registers.
a) 00111 b) 11001 c) 00000

Ans: 00000111
11111001
00000000

## * Unsigned Numbers addition:

Ex: 1) 1 0 1 0 = A = 10
0 0 1 1 = 3
1 1 0 1 = 13 = D
} Carry case

2)
1 1 0 0 = 12
0 1 0 1 = 5
1 0 0 0 1 = 1
} overflow case

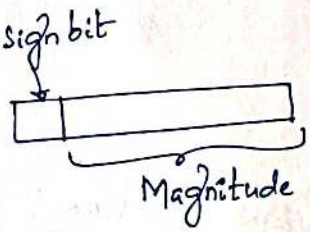
overflow → 1 (carry)

3)
1 ← This carry is not an overflow. It has to be carried forward to the other half.
1 0 1 0 | 1 1 0 0
0 0 1 1 | 0 1 0 1
1 1 1 0 | 0 0 0 1
} Ignoring overflow.

## * Sign - Magnitude Representation:

If the Sign bit is 0, the number is +ve.

If the Sign bit is 1, the Number is -ve.




128 64 32 16 8 4 2 1

Ex: +18 = 0 0 0 1 0 0 1 0

-18 = 1 0 0 1 0 0 1 0 (Sign magnitude)

Note
$18_{10}$ = 12H = 00010010

## * Drawbacks of Sign - magnitude Representation:

1. Addition and Subtraction requires a consideration of both the signs of the numbers and relative magnitudes to carry out the required operation.

2. There are two Representations of 0:

$+0_{10}$ = 0000 0000

$-0_{10}$ = 1000 0000 (Sign Magnitude)

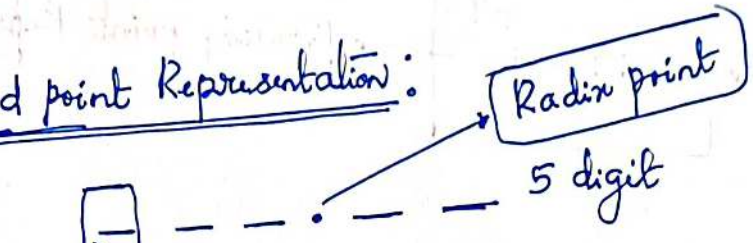
- Because of the above drawbacks, Sign - magnitude representation is rarely used in implementing the integer portion of ALU. Instead, the most common scheme is two's complement Representation.

Dr. KBR

## Topics discussed ②

1) Binary Arithmetic
   - fixed point:
   - floating point: current Architectures follows this method

### * Fixed point Representation:




2 3 . 4 5

\+ 0 0 . 0 0

− 0 0 . 0 0

− 0 0 . 5 0 → -0.5

\+ 0 0 . 0 0 → 0.0001 → This representation is not possible

↑ we can't represent this number

-99.99 to 99.99 → Range of Representation

0.0001 → 0.00 → Reducing the precision

### 2) Floating point Representation:

Example: = 2 0 1 5

Sign | Exp | Mantissa

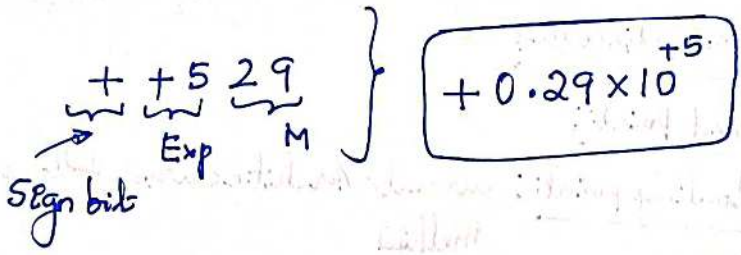
$(0.M) \times B^{Exp.}$ → Base

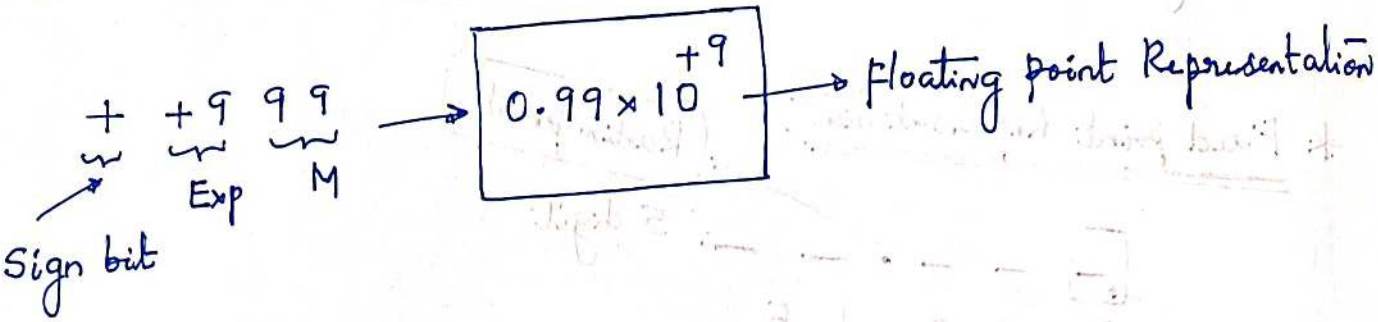
$-0.15 \times 10^{20}$

Example: − − 5 2 7

Sign | Exp | Mantissa

$-0.27 \times 10^{-5}$




In fixed point Representation, +99.99
- 99 → Integer part
- . → Radix point
- 99 → fractional part

→ Sign Extension

→ Sign - Magnitude Representation

Ex: +18 = 00010010
-18 = 10010010 [Sign magnitude]

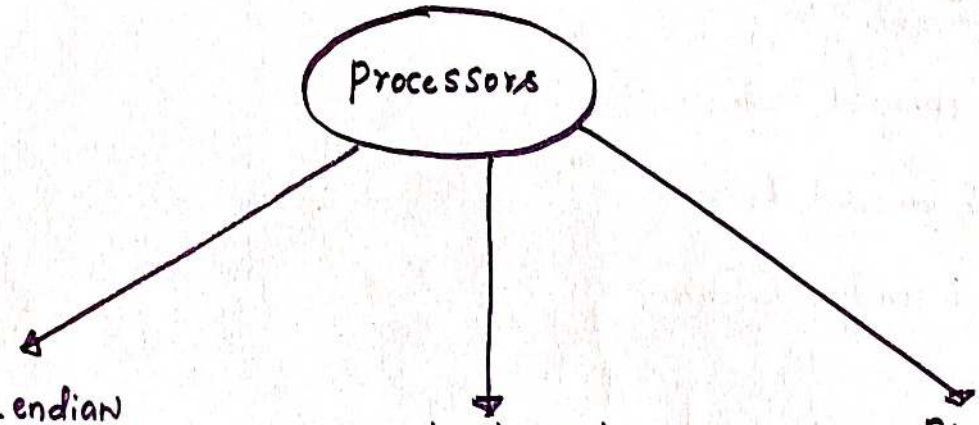
→ The 16-bit unsigned binary Representation of 152:

0000 0000 10011000 = 152

→ The 16-bit signed binary Representation of -152:

1000 0000 10011000 = -152

# INSTRUCTION SETS: Characteristics and Functions




**Big-endian**

- The most Significant byte in the Lowest numerical address

**Little-endian**

- The most Significant byte is stored in the highest numerical address.

**Bi-endian**

- A bi-endian processor can handle both styles.

## Machine instruction set

**Computer Designer**

- The machine instruction set provides the functional requirements for cpu.
- The implementation of cpu involves the implementation of machine instruction set.

**Computer programmer**

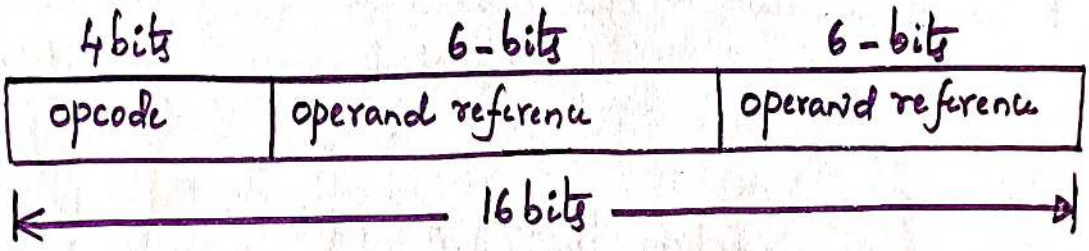
- The programmer must be aware of register, memory structure and the types of data directly supported by the machine and the functioning of the ALU.

* The operation of the cpu is determined by the instructions it executes, referred to as machine instructions or computer instructions. The collection of different instructions that the cpu can execute is referred to as the cpu's instruction set.

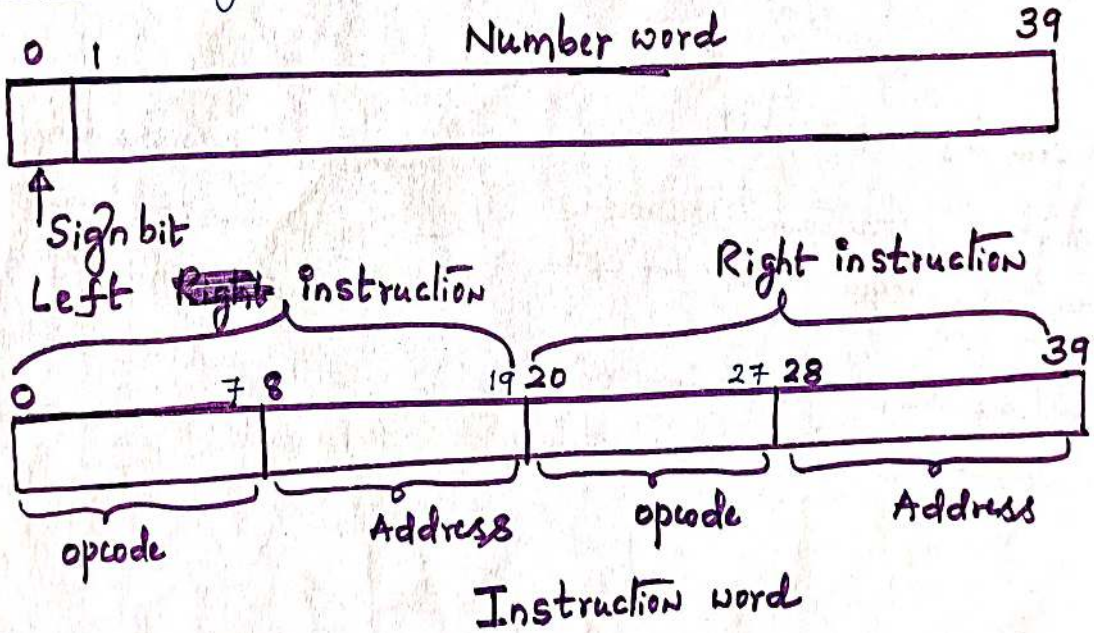
## * Elements of a machine Instruction :

1) Operation code
2) Source operand reference
3) Result operand Reference
4) Next instruction Reference

## * Simple example of an Instruction format :




IAS Memory Formats




## * Simple examples of Mnemonics :

ADD — Add
SUB — Subtract
MPY — Multiply
DIV — Divide
LOAD — Load data from memory
STOR — store data to memory

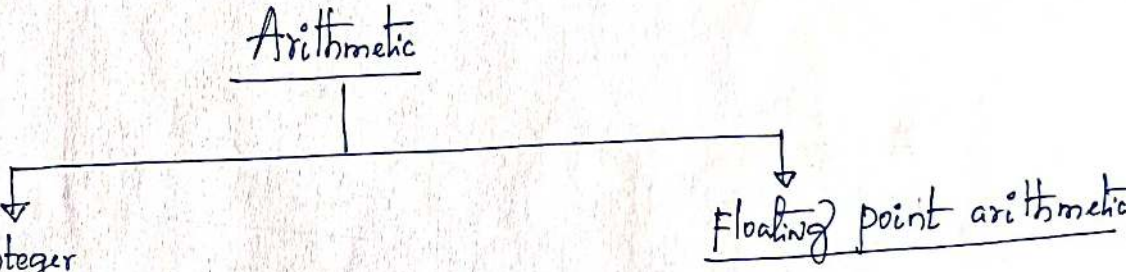
# Computer Arithmetic

* objectives:

1) The way in which numbers are represented (the binary format).

2) The algorithms used for the basic arithmetic operations (add, subtract, multiply, divide)

- The above two considerations/objectives apply both to integer and floating point arithmetic.




* The decimal point is always assumed to be to the right of the Least Significant digit.

OR

* Assumed to be to the right of the right most digit.

* Floating point numbers can be used to represent very Large and very small numbers.

* Most processors implement the IEEE 754 Standard for floating point representation and floating-point arithmetic.

* The position of binary point is variable. i.e the binary point is said to float and the numbers are called floating-point numbers.

Dr. K. B. Ramesh

## * Floating Point Representation:

(P) Represent $1259.125_{10}$ in Single precision and double precision formats.

Solution: Step1 Convert decimal number into binary format

| 16 | 1259 |
|---|---|
| 16 | 78 — 11 |
| | 4 — 14 |

78 → 4

| 16 | 1259 |
|---|---|
| | 112 |
| | 139 |
| | 128 |
| | 11 — B |

| 16 | 78 |
|---|---|
| | 64 |
| | 14 — E |

$1259_{10} = 4EBH$

$= 0100\ 1110\ 1011$

Fractional part:

$0.125 \times 2 = 0.25 \quad 0$

$0.25 \times 2 = 0.5 \quad 0$

$0.5 \times 2 = 1.0 \quad 1$

$.125_{10} = 0.001$

NOTE Verification

0100 1110 1011 =

| 1024 | 512 | 256 | 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |
|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 0 | 0 | 1 | 1 | 1 | 0 | 1 | 0 | 1 | 1 |

$1024 + 128 + 64 + 32 + 8 + 2 + 1 = 1259$

Fractional part

$.0\ 0\ 1 = 0 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} = \frac{1}{8}$

$= 0.125$

∴ Binary Number = 100 1110 1011 . 001

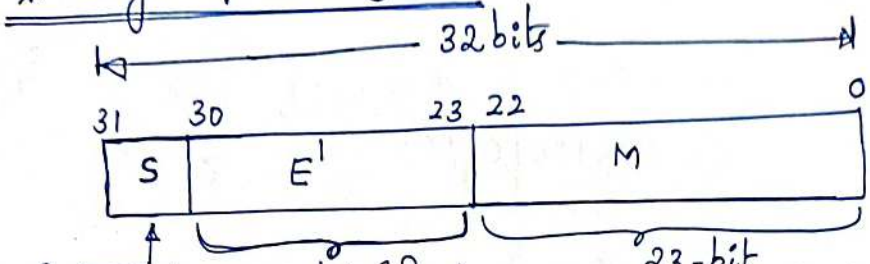
Step2: Normalize the Number

$1.0011101011001 \times 2^{10}$

Step3: Single precision Representation

S=0, E=10, M=00111010111001

## * Single - precision format




Sign of number
0 → +ve
1 → -ve

8-bit Signed Exponent in excess-127 Representation

23-bit Mantissa fraction

∴ Bias = 127 for Single precision formal

∴ $E' = E + Bias$

$= 10 + 127$

$E' = 137_{10}$

$= 10001001_2$

Note:

| 1 | 0 | 0 | 0 | 1 | 0 | 0 | 1 |
|---|---|---|---|---|---|---|---|
| 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |

← weightage

$128 + 8 + 1 = 137_{10}$

∴ The Number in Single precision format is given as

0 | 10001001 | 00111010111001.....0

Sign | Exponent | Mantissa (23-bits)

## * Double precision Representation :

For a given Number

$S=0, \quad E=10, \quad M=0011101011001$

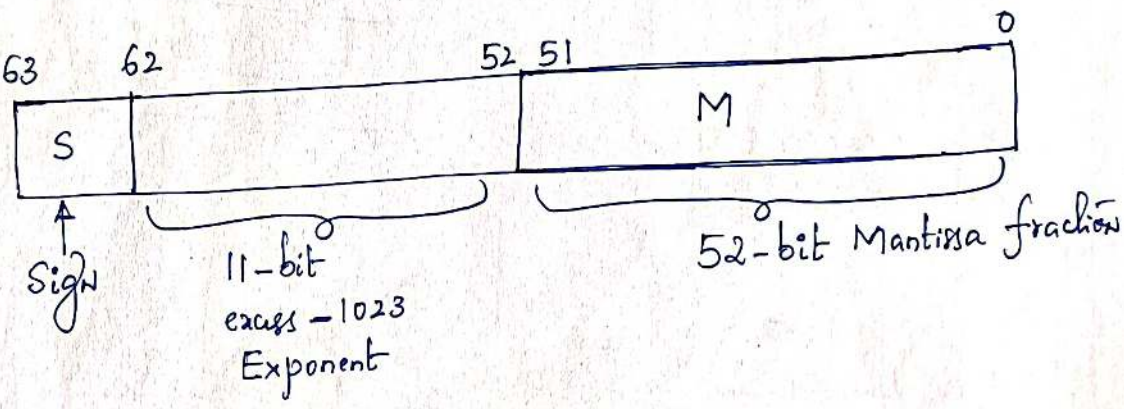
$\therefore E' = E + Bias$

$= 10 + 1023$ $\quad$ Bias = 1023 for Double precision format

$= 1033$

$= 10000001001_2$

### - Double precision format




| 1024 | 512 | 256 | 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |
|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 1 |

$\Rightarrow 1024 + 8 + 1 = 1033$

$\therefore$ The Number in double precision format

0 | 10000001001 | 0011101011001 ..... 0

Sign | Exponent | Mantissa (52-bits)

(P) Represent $-307.1875_{10}$ in Single precision and double precision formats.

Solution:

Step1: Convert decimal Number into binary format

```
      19          1
   ------      ------
16)307      16)19
   16          16
   ---         ---
   147         3
   144
   ---
     3
```

```
16 | 307
16 | 19 - 3
     1 - 3
```

$\therefore 307_{10} = 133H = 0001\ 0011\ 0011$

Fractional part:

$0.1875 \times 2 = 0.3750 \quad 0$

$0.3750 \times 2 = 0.750 \quad 0$

$0.75 \times 2 = 1.5 \quad 1$

$0.5 \times 2 = 1.0 \quad 1$

$0$

$\therefore 0.1875 = 0.0011$

Verification

| 1024 | 512 | 256 | 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 |

$= 256 + 32 + 16 + 2 + 1$

$= 307_{10}$

$0.0011 = 0 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} + 1 \times 2^{-4}$

$= \frac{1}{8} + \frac{1}{2^4} = \frac{2+1}{16} = \frac{3}{16}$

$= 0.1875$

$\therefore -307.1875_{10} = -100110011.0011$

Step2: Normalize the Number

$-1.00110011 0011 \times 2^8$

Step3: Single precision Representation

$S = -1$; $E = 8$; $M = 00110011 0011$

$\therefore E' = E + 127 = 8 + 127 = 135_{10}$

$= 10000111_2$

$\therefore$ 1 | 10000111 | 00110011001100....0

S | E | M (23-bits)

Dr. KBR

Step4 : Double precision Representation

For a given Number,

S=1, E=8 M = 0011 0011 0011

$\therefore E' = E + 1023 = 8 + 1023 = 1031_{10}$

| 1024 | 512 | 256 | 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | $= 1024+4+2+1$ |

$= 1031_{10}$

$\therefore E' = 10000000111_2$

∴ Number in double precision format is

1 | 10000000111 | 0011 0011 0011 0 ..... 0

Sign | Exponent | Mantissa (52 bits)

(P) Represent $0.0625_{10}$ in Single precision and double precision formats.

Solution:

Step1 Convert decimal number into binary format

Integer part: 0

Fractional part:

$0.0625 \times 2 = 0.125$ 0

$0.125 \times 2 = 0.25$ 0

$0.25 \times 2 = 0.5$ 0

$0.5 \times 2 = 1.0$ 1

0

$\therefore$ 0.0625 = 0.0001

Note:

$0 \times 2^{-1} + 0 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4}$

$= \frac{1}{2^4} = \frac{1}{16} = 0.0625_{10}$

Step2: Normalize the Number

$1.0 \times 2^{-4}$

Step3: S=0, E=-4 M=0 [Single precision Representation]

$E' = E + 127 = -4 + 127 = 123_{10}$

| 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |
|---|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 1 | 1 | 0 | 1 | 1 |

$123_{10} = 01111011_2$

$\therefore$ 0 01111011 0000.....0

Sign Exponent 23 bits

**Step4 : Double precision Representation**

$E' = E + 1023$

$= -4 + 1023$

$= 1019_{10}$

| 1024 | 512 | 256 | 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 1 |

$= 512 + 256 + 128 + 64 + 32 + 16 + 8 + 2 + 1$

$= 1024$

$\therefore$ 0 01111111011 0000.....0

Sign | Exponent | Mantissa (52-bits)

## * Floating point Representation:

Ex:

$976,000,000,000,000 \longrightarrow 9.76 \times 10^{14}$

$0.0000000000000976 \longrightarrow 9.76 \times 10^{-14}$

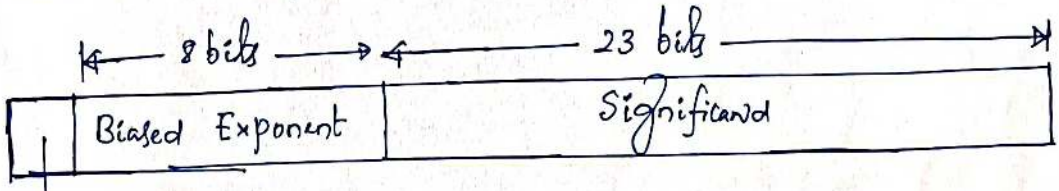
Form: $\pm S \times B^{\pm E}$

Sign: plus or minus.

Significand S.

Exponent E.

Format: Binary floating point Numbers.




Example: 0 10010011 10.10001000000000000000 $= 1.638125 \times 2^{20}$

128 64 32 16 8 4 2 1

$2^{-1}\ 2^{-2}\ 2^{-3} \quad 2^{7}$

$128 + 16 + 2 + 1 = 147$

$2^{-1} + 2^{-3} + 2^{-7}$

$\frac{1}{2} + \frac{1}{8} + \frac{1}{2^7} = \frac{1}{2} + \frac{1}{8} + \frac{1}{128}$

$= \frac{64 + 16 + 1}{128} = \frac{81}{128}$

Bias + power

$127 + 20 = 147$

Biased Exponent $= 2^{K-1} - 1 = 2^{8-1} - 1 = 127$

## * ~~Two's complement Representation~~ :

Example:

* The 16-bit unsigned binary Representation of 152.

128 64 32 16 8 4 2 1

0000 0000 1001 1000 = 152

128
16
8
0
152

* The 16-bit Signed binary Representation of -152.

1000 0000 10011000

Sign bit

Example:

* Multiply the Numbers +7 and -5, using 6-bit Signed-magnitude Integers.

Solution: +7 = 000111

-5 = 100101

-35 = 1100011

( 32 16 8 4 2 1

| 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |
|---|---|---|---|---|---|---|---|
| 0 | 1 | | 0 | 0 | 0 | 1 | 1 |

## * Two's complement Notation:

### - Signed Two's complement Addition:

Ex: Five bit ALU including the Sign bit

As per the 2's complement Representation, positive numbers are represented without any change (in true binary) with a sign bit of 0. The negative numbers have sign bit of 1 and their magnitudes are converted into 2's complement form. All the bits including the sign bits take part in addition process.

1) $+7 = 00111$ → $00111$

$-3 = 10011$ → 1's complement $11100$

$+1 \quad 00001$

$11101$ → $11101$

$\boxed{1}\,00100$

| 16 | 8 | 4 | 2 | 1 |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 0 |

$+4$

2) $-6 = 10110$ → 1's com → $11001$

$+1 \quad 00001$

$11010$ → $11010$

$+5 = 00101$ → $00101$

$-1$ → $11111$ → The Result is negative and in 2's complement form

$10000$

$+ \quad 1$

$10001$

3) ~~$+7$~~ $5$

~~$-2$~~

3) $+9 = 01001$

$+8 = 01000$

$17$ → $10001$ (overflow)

4) $-9 = 11001$ → 1's → $10110$ → $+1$ → $10111$

$-3 = 10011$ → 1's → $11100$ → $+1$ → $11101$

$-12$ → $\boxed{1}\,10100$ → The Result is negative in 2's complement form

$11011$

$\quad\quad 1$

$11100$

5) $-9 = 11001$ → 1's → $10110$ → $+1$ → $10111$

$-8 = 11000$ → 1's → $10111$ → $+1$ → $11000$

$\boxed{1}\,01111$

C ↓ NC ↓ 1's

$00000$

$+1 \quad\quad 1$

$00001$

There is no carry from the MSB position but the Sign bit position has a carry.

This case is a example of overflow.

6) $-9 \longrightarrow 11001 \xrightarrow{1's} 10110 \xrightarrow{+1} 10111$

$+9 \longrightarrow 01001 \longrightarrow 01001$

$\boxed{1}\ 00000$

The Rule for overflow:

- If two numbers are added, and they are both positive or both negative, then overflow occurs if and only if the result has the opposite sign.

Example:

* Add the Quantities +3 and −4 in 4-bit two's complement Notation

8 4 2 1

$-4 = 1100 \xrightarrow{1's} 1011 \xrightarrow{+1} 1100$

$+3 = 0011 \longrightarrow 0011$

$-1 \qquad 1111$ → is the two's complement representation of −1.

$1000$

$+\quad 1$

$1001$

* Compute −3−4 in 4-bit Two's Complement Notation

$-4 = 1100 \xrightarrow{1's\ comp.} 0011 \xrightarrow{+1} 0100 \quad 1011 \xrightarrow{+1} 1100$

$-3 = 1011 \xrightarrow{1's\ comp.} 0100 \xrightarrow{+1} 0101 \quad 1100 \xrightarrow{+1} 1101$

$-7 \qquad \boxed{1}\ 1001$

The two's complement representation of −7

$1001$

$\downarrow 1's$

$1110$

$+\quad 1$

$\boxed{1}\ 1111$

$-\ 7$

p) what is the 16-bit Sign-magnitude Representation of the 8-bit Sign magnitude Quantity 10000111 (-7)?

Solution: 1000000000000111

~~p) what is the 16-bit Sign extension of the 8-bit~~

p) using 8-bit two's complement Integers, perform the following computations

$-35 + (-11)$

Note

128 64 32 16 08 4 2 1

$-35 = 10100011$ — 1's complement → $11011100$

$+1$ → $11011101$

$-11 = 10001011$ — 1's complement → $11110100$

$+1$ → $00000001$

$11110101$

$11011101$ (+)

$\boxed{1}\ 11010010$

1's complement

$00101001$

$+1$

$00101010$

$32\ 16\ 8\ 4\ 2\ 1$

$-46$

19 − (−4)
↳ Subtraction

Case1: 19 − (−4) = 19 + 4

19 = 0 0 0 1 0 0 1 1
4 = 0 0 0 0 0 1 0 0
23 = 0 0 0 1 0 1 1 1 → 1, 2, 4, 16

128 64 32 16 8 4 2 1

Case2: 19 − (−4) [Subtraction]

19 = 0 0 0 1 0 0 1 1 → 1's → 1 1 1 0 1 1 0 0 → +1 → 1 1 1 0 1 1 0 1
−4 = 1 0 0 0 0 1 0 0 → 1's → 0 1 1 1 1 0 1 1 → +1 → 0 1 1 1 1 1 0 0

19 = 0 0 0 1 0 0 1 1 → 0 0 0 1 0 0 1 1
−4 = 1 0 0 0 0 1 0 0 → 1's → 1 1 1 1 1 0 1 1 → +1 → 1 1 1 1 1 1 0 0

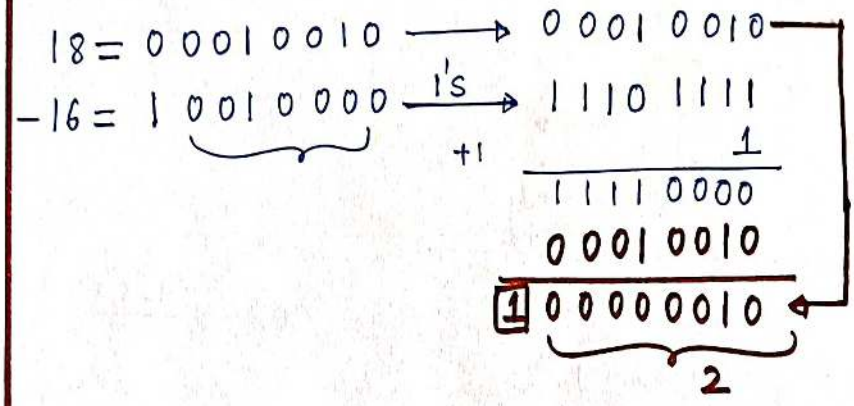
↓ Negate it by 2's complement

1 0 0 0 0 1 0 0
1 1 1 1 1 0 1 1
1
1 1 1 1 1 1 0 0

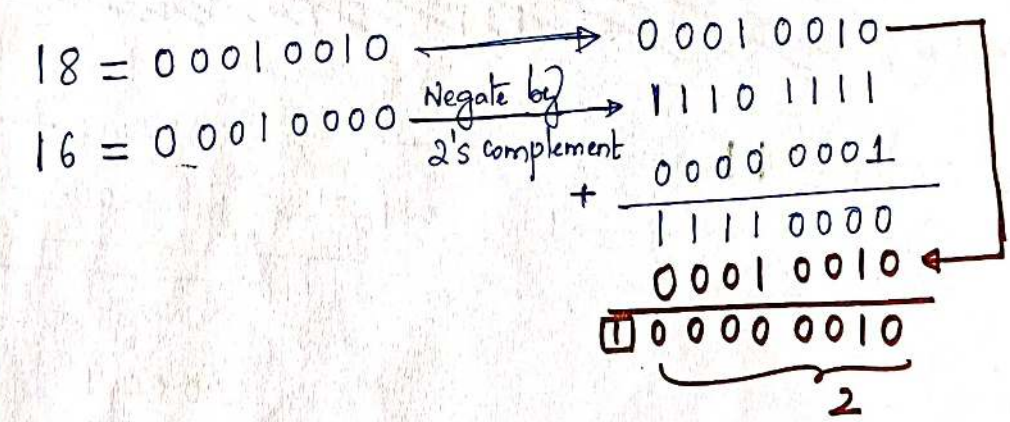
0 0 0 0 0 0 1 1
0 0 0 1 0 0 1 1 (+)
0 0 0 1 0 1 1 0
+ 0 0 0 0 0 0 0 1
0 0 0 0 0 1 0 0
0 0 0 1 0 0 1 1 (+)
0 0 0 1 0 1 1 1 → 1, 2, 4, 16
23

<P> Using 8-bit two's complement integers, show how to perform the operation 18 − 16.

Method 1: 18 + (−16) = 2 (Addition)




Method 2: 18 − 16 = 2 (Subtraction)




Note: To subtract one number (subtrahend) from another (minuend), take the two's (negation) of the subtrahend and add it to the minuend.

# * Hardware Implementation of unsigned Binary Multiplication

**Example:**

11 × 13 → Multiplicand

33 ← Multiplier

11

143

A — 10 → 1010
B — 11 → 1011
C — 12 → 1100
D — 13 → 1101

1011 × 1101

1011
0000
1011
1011
10001111

128 64 32 16 8 4 2 1 ⇒ 128 + 8 + 4 + 2 + 1 = 143

**NOTE:**

* A, Q, M and C are the registers.
* C → 1-bit register.

**Implementation:**

P1)

| C | A | Q | M | |
|---|---|---|---|---|
| 0 | 0000 | 1101 | 1011 | Initial values |

Add: 1011 − (M), 0000 − (A) ⇒ 1011 − (A)

I cycle:
0 1011 1101 — 1011 Add
0 0101 1110 — 1011 shift

II cycle:
0 0010 1111 — 1011 shift

Add: 1011 − (M), 0010 − (A) ⇒ 1101 − (A)

III cycle:
0 1101 1111 — 1011 Add
0 0110 1111 — 1011 shift

Fourth cycle (IV):
1 0001 1111 — 1011 Add
0 1000 1111 — 1011 shift

Add: 1011 − (M), 0110 − (A) ⇒ 10001 − (A)

Result

**NOTE:** if $Q_0 = 1$ then Add operation
if $Q_0 = 0$ then no add operation.

**NOTE:** The process is repeated for each bit of the original multip

P2)

13 × 11 → Multiplier
↓
Multiplicand

| | | Multiplier | Multiplicand | |
|---|---|---|---|---|
| C | A | Q | M | |
| 0 | 0000 | 1011 | 1101 | Initial values |
| 0 | 1101 | 1011 | 1101 | Add (M)=1101, (A)=0000 |
| 0 | 0110 | 1101 | 1101 | Shift → (A) = 01101 |
| I cycle | | | | |
| 1 | 0011 | 1101 | 1101 | Add |
| 0 | 1001 | 1110 | 1101 | shift → (M)=1101, (A)=0110 → (A)=10011 |
| II cycle | | | | |
| 0 | 0100 | 1111 | 1101 | Shift |
| III cycle | | | | |
| 1 | 0001 | 1111 | 1101 | Add (M)=1101, (A)=0100 |
| 0 | 1000 | 1111 | 1101 | shift → 10001 |
| IV cycle | | | | |

Result

Operation

- The Hardware consists of Registers A, Q, M and the carry flag C.
- The Register Q is loaded with the multiplier.
- The Register M is loaded with the multiplicand
- Initially, the carry flag C and Register A are cleared to 0.
- At the start, based on the bit value at $Q_0$ in the Q register, the control sequencer generates the Add/No Add signal

- The add signal makes the content of Register A to be added with the multiplicand M and this addition is performed by n-bit adder Logic.

- After Add/No Add, the register C, A and Q are shifted one bit to right. This completes one cycle.

- The same steps is repeated for 'n' cycles.

- At the end of n cycle the content of Register A and Q is the result of the multiplication.

## * Booth's Algorithm :-

- A powerful algorithm for Signed-number multiplication.

- It works with any combination of positive and negative numbers.

Example: $7 \times 3 = 21$

7 → Multiplier
3 → Multiplicand

| | A | Q (Multiplier) | $Q_{-1}$ | M (Multiplicand) | |
|---|---|---|---|---|---|
| | 0000 | 0011 | 0 | 0111 | Initial values |
| I cycle | 1001 | 0011 | 0 | 0111 | $(A) \leftarrow A - M$ |
| | 1100 | 1001 | 1 | 0111 | shift |
| II cycle | 1110 | 0100 | 1 | 0111 | shift |
| III cycle | 0101 | 0100 | 1 | 0111 | $(A) \leftarrow A + M$ |
| | 0010 | 1010 | 0 | 0111 | shift |
| IV cycle | 0001 | 0101 | 0 | 0111 | shift |

Result

$Q_0\ Q_{-1}$ = 1 0 → $(A) - (M) \rightarrow (A)$ then Right shift
(A) = 0000
(M) = 0111
1001

$Q_0\ Q_{-1}$ = 0 1 → $(A) + (M) \rightarrow (A)$ then Right shift
(A) = 1110
(M) = 0111
1 0101

(A) = 0000
(M) = 0111
1000
+ 1
1001
0000
1001

| 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 | |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 0 | 1 | 0 | 1 | $= 16 + 4 + 1 = 21$ |

NOTE

| $Q_0$ | $Q_{-1}$ | operation |
|---|---|---|
| 1 | 0 | $(A) - (M) \rightarrow (A)$ ; Then Right shift |
| 0 | 1 | $(A) + (M) \rightarrow (A)$ ; Then Right shift |
| 1 | 1 | only Shift Right |
| 0 | 0 | only Shift Right |

(P) $7 \times -3 = 21$

7 → Multiplicand

-3 → Multiplier = 1011 (8 4 2 1)

1's → 01100 + 1 = 1101

| A | Q (Multiplier) | $Q_{-1}$ | M | |
|---|---|---|---|---|
| 0000 | 1101 | 0 | 0111 | Initial Values |
| 1001 | 1101 | 0 | 0111 | (A) ← (A) − (M) |
| 1100 | 1110 | 1 | 0111 | Shift |
| 0011 | 1110 | 1 | 0111 | (A) ← (A) + (M) |
| 0001 | 1111 | 0 | 0111 | Shift |
| 1010 | 1111 | 0 | 0111 | (A) ← (A) − (M) |
| 1101 | 0111 | 1 | 0111 | shift |
| 1110 | 1011 | 1 | 0111 | shift |

1001 0100 + 1 = 1001 0101 ⇒ 16 + 04 + 01 = 21

(16 8 4 2 1)

= −21

$Q_0$ $Q_{-1}$: 1 0

1) (A) − (M) → (A)

(A) = 0000

(M) = 0111

→ 1000 + 1 = 0001

1001

0000

1001

$Q_0$ $Q_1$: 0 1

2) (A) + (M) → (A)

1100 (A)

0111 (M)

1 0011

3) (A) − (M) → (A)

0001

− 0111

→ 1000 + 1 = 1001

0001

1010

4) $Q_0$ $Q_{-1}$: 1 1

Shift only

<P>

$-5 \times 4$ (Multiplier)

$-5 = 1101$ is $\rightarrow 1010 + 0001 = 1011$ (Multiplicand)

| A | Q (Multiplier) | $Q_{-1}$ | M | |
|---|---|---|---|---|
| 0000 | 0100 | 0 | 1011 | Initial Value |
| 0000 | 0010 | 0 | 1011 | shift Right |
| 0000 | 0001 | 0 | 1011 | shift Right |
| 0101 | 0001 | 0 | 1011 | $(A)-(M) \rightarrow (A)$ |
| 0010 | 1000 | 1 | 1011 | Shift |
| 1101 | 1000 | 1 | 1011 | $(A)+(M) \rightarrow (A)$ |
| 1110 | 1100 | 0 | | |

$Q_0\ Q_{-1}$ : 1 0

$(A)-(B) \rightarrow (A)$

$(A) = 0000$

$(M) = 1011$

$0100 + 0001 = 0101$

$(A)\ 0000 = 0101$

$Q_0\ Q_{-1}$ : 0 1

$(A)+(M) \rightarrow (A)$

$(A) = 0010$

$(M) = 1011$

$1101$

$1110\ 1100$

$1001\ 0011$

$+1 \quad\quad 1$

$1001\ 0100 = 16+4 = 20$

$-20$

## * Division :

7 → Dividend → Q Register

3 → Divisor → M Register

$$3\overline{)7}\quad 2 \text{ — Quotient},\ 6,\ 1 \text{ — Remainder}$$

M = 0011

| A | Q | | |
|---|---|---|---|
| 0000 | 0111 | Initial Value | A & M Signs are Same |
| 0000 | 1110 | Left Shift | (A) = 0000 |
| 1101 | | Subtract | − (M) = 0011 |
| | | | (A) = 1101 |
| 0000 | 1110 | Restore | Sign bit of A = 1 then $Q_0 = 0$ |
| | | | The sign of A is different before and after the operation |
| 0001 | 1100 | Shift | (A) = 0001 |
| 1110 | | Subtract | − (M) = 0011 |
| | | | (A) = 1110 |
| 0001 | 1100 | Restore | Sign bit of A = 1 then $Q_0 = 0$ |
| 0011 | 1000 | Shift | (A) = 0011 |
| 0000 | | Subtract | − (M) = 0011 |
| | | | (A) = 0000 |
| 0000 | 1001 | Set $Q_0 = 1$ | The Sign of A is same before and after the operation or (A) = 0 then $Q_0 = 1$ |
| 0001 | 0010 | Shift | (A) = 0001 A & M Signs are same |
| 1110 | | Subtract | − (M) = 0011 |
| | | | (A) = 1110 |
| 0001 | 0010 | Restore | Sign bit of A = 1 then $Q_0 = 0$ |
| Remainder | Quotient | | |

NOTE: If M and A have the same signs, then (A) ← (A) − (M)

If M and A have the different signs, then (A) ← (A) + (M)

(P) Suppose an 8-bit data word stored in memory is 11000010. using Hamming algorithm, determine what check bits would be stored in memory with data word. Suppose that data bit 3 sustains an error and is changed from 0 to 1. what are the new check bits? Show how you got your answer.

| $D_8$ | $D_7$ | $D_6$ | $D_5$ | $D_4$ | $D_3$ | $D_2$ | $D_1$ |
|---|---|---|---|---|---|---|---|
| 1 | 1 | 0 | 0 | 0 | 0 | 1 | 0 |

Solution:

| Bit position | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Position Number | 1100 | 1011 | 1010 | 1001 | 1000 | 0111 | 0110 | 0101 | 0100 | 0011 | 0010 | 0001 |
| Data bit | $D_8$ | $D_7$ | $D_6$ | $D_5$ | | $D_4$ | $D_3$ | $D_2$ | | $D_1$ | | |
| Check bit | | | | | $C_8$ | | | | $C_4$ | | $C_2$ | $C_1$ |

Check bits: $C_1 = D_1 \oplus D_2 \oplus D_4 \oplus D_5 \oplus D_7 = 0 \oplus 1 \oplus 0 \oplus 0 \oplus 1 = 0$

$C_2 = D_1 \oplus D_3 \oplus D_4 \oplus D_6 \oplus D_7 = 0 \oplus 0 \oplus 0 \oplus 0 \oplus 1 = 1$

$C_4 = D_2 \oplus D_3 \oplus D_4 \oplus D_8 = 1 \oplus 0 \oplus 0 \oplus 1 = 0.$

$C_8 = D_5 \oplus D_6 \oplus D_7 \oplus D_8 = 0 \oplus 0 \oplus 1 \oplus 1 = 0.$

0010

| $D_8$ | $D_7$ | $D_6$ | $D_5$ | $D_4$ | $D_3$ | $D_2$ | $D_1$ |
|---|---|---|---|---|---|---|---|
| 1 | 1 | 0 | 0 | 0 | 1 | 1 | 0 |

$C_1 = 0$

$C_2 = 0 \oplus 1 \oplus 0 \oplus 0 \oplus 1 = 0$

$C_4 = 1 \oplus 1 \oplus 0 \oplus 1 = 1$

$C_8 = 0$

0100

∴

0010

0100 ⊕

0110

Bit position

Data bit $D_3$

Indicates that bit position 6 which contains data bit 3 is in error

## * Subtraction of Numbers in two's complement Representation (M-S)

M – Minuend

S – Subtrahend

8 4 2 1
0 0 1 0

1) 2 - 7

2 = 0 0 1 0 → 0 0 1 0

7 = 0 1 1 1 —Negate by 2's complement→ 1 0 0 0 —+1→ 1 0 0 1

-5

1 0 1 1 .

↓ 1's

1 1 0 0

1

1 1 0 1

-5

2) 5 - 2

5 = 0 1 0 1 → 0 1 0 1

2 = 0 0 1 0 —Negate by 2's complement→ 1 1 0 1 —+1→ 1 1 1 0

3

1 0 0 1 1

3

3) -5 - 2

-5 = 1 1 0 1 —1's→ 1 0 1 0 —+1→ 1 0 1 1

-2 = 0 0 1 0 —Negate→ 1 1 0 1 —+1→ 1 1 1 0

-7

1 1 0 0 1

↓ 1's complement

1 1 1 0

\+ 1

1 1 1 1

-7

4) $5 + 2 = 7 \Rightarrow 5 - (-2) = 7$

$5 = 0101 \longrightarrow 0101 \longrightarrow 0101$

~~$-2 = 1010 \xrightarrow{\text{Negation}} 0101 \xrightarrow{+1} 0110$~~

$-2 = 1010 \xrightarrow{1's} 1101 \xrightarrow{+1} 1110 \xrightarrow{\text{Negation}} 0001 \xrightarrow{+1} 0010$

$0101 + 0010 = 0111$

$0111 \rightarrow 7$

5) $7 - 7$

$7 = 0111 \longrightarrow 0111$

$7 = 0111 \xrightarrow{\text{Negation}} 1000 \xrightarrow{+1} 1001$

$0111 + 1001 = 10000$

6) $7 + 7 = 7 - (-7)$

$7 = 0111 \longrightarrow 0111$

$-7 = 1111 \xrightarrow{1's} 1000 \xrightarrow{+1} 1001 \xrightarrow{\text{Negate}} 0110 \xrightarrow{+1} 0111$

$0111 + 0111 = 1110$ (overflow)

$1001 + 1 = 1010$

7) $-6 - 4$

$-6 = 1110 \xrightarrow{1's} 1001 \xrightarrow{+1} 1010$

$-4 = 1100 \xrightarrow{1's} 1011 \xrightarrow{+1} 1100$

$1010 + 1100 = 10110$ (overflow)

$0110 \xrightarrow{1's} 1001 \xrightarrow{+1} 1010$

$1001 + 1 = 1010$

## * Integer Multiplication :

- Integer multiplication is more complex than addition.

- Multiplication of unsigned Numbers :

Manual method →

Multiplicand Multiplier

1010 × 1101

1010
0000
1010
1010 } partial products

10000010

130

$\frac{10 \times 13}{130}$

128 64 32 16 8 4 2 1

1 0 0 0 0 0 1 0

130.

* ~~Multiplication~~ The above procedure is slightly modified while implementing in a computer so that multiplication is done efficiently by hardware

## * Booth's Algorithm :

- The Booth's algorithm handles both +ve and -ve 2's complement operands without need for any correction in the procedure or result.

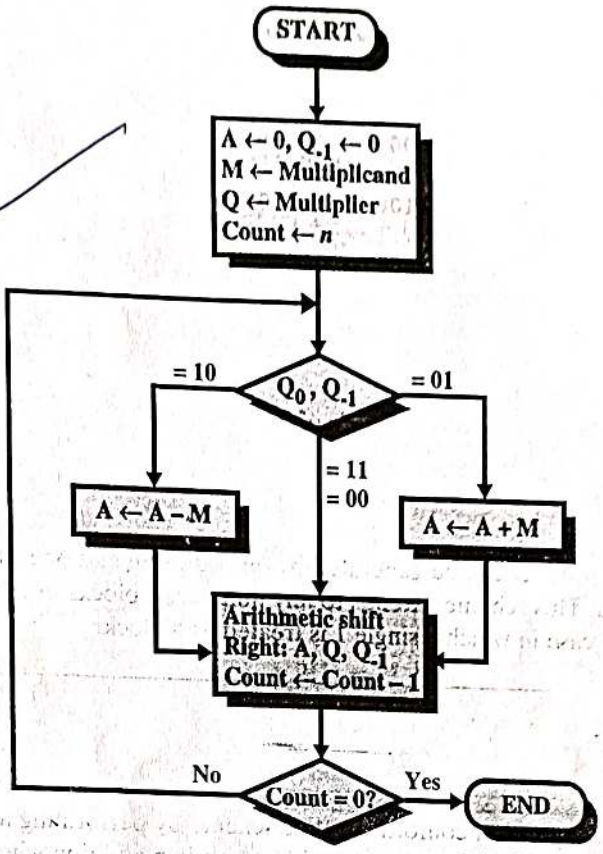



**Figure 9.12** Booth's Algorithm for Twos Complement Multiplication

$$M \times (00011110) = M \times (2^4 + 2^3 + 2^2 + 2^1)$$
$$= M \times (16 + 8 + 4 + 2)$$
$$= M \times 30$$

The number of such operations can be reduced to two if we observe that

$$2^n + 2^{n-1} + \ldots + 2^{n-K} = 2^{n+1} - 2^{n-K} \qquad (9.3)$$

$$M \times (00011110) = M \times (2^5 - 2^1)$$
$$= M \times (32 - 2)$$
$$= M \times 30$$

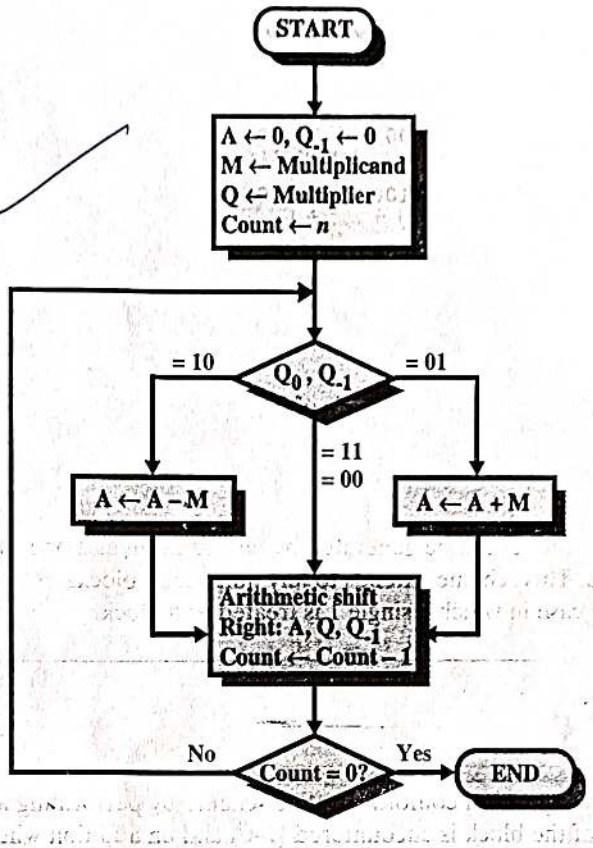



**Figure 9.12** Booth's Algorithm for Twos Complement Multiplication

$$M \times (00011110) = M \times (2^4 + 2^3 + 2^2 + 2^1)$$
$$= M \times (16 + 8 + 4 + 2)$$
$$= M \times 30$$

The number of such operations can be reduced to two if we observe that

$$2^n + 2^{n-1} + \ldots + 2^{n-K} = 2^{n+1} - 2^{n-K} \qquad (9.3)$$

$$M \times (00011110) = M \times (2^5 - 2^1)$$
$$= M \times (32 - 2)$$
$$= M \times 30$$

| A | Q | $Q_{-1}$ | M | | |
|---|---|---|---|---|---|
| 0000 | 0011 | 0 | 0111 | Initial values | |
| 1001 | 0011 | 0 | 0111 | $A \leftarrow A - M$ | First cycle |
| 1100 | 1001 | 1 | 0111 | Shift | First cycle |
| 1110 | 0100 | 1 | 0111 | Shift | Second cycle |
| 0101 | 0100 | 1 | 0111 | $A \leftarrow A + M$ | Third cycle |
| 0010 | 1010 | 0 | 0111 | Shift | Third cycle |
| 0001 | 0101 | 0 | 0111 | Shift | Fourth cycle |

**Figure 9.13** Example of Booth's Algorithm (7 × 3)

So the product can be generated by one addition and one subtraction of the multiplicand. This scheme extends to any number of blocks of 1s in a multiplier, including the case in which a single 1 is treated as a block.

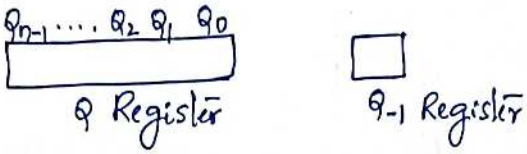
$$M \times (01111010) = M \times (2^6 + 2^5 + 2^4 + 2^3 + 2^1)$$
$$= M \times (2^7 - 2^3 + 2^2 - 2^1)$$

Booth's algorithm conforms to this scheme by performing a subtraction when the first 1 of the block is encountered (1–0) and an addition when the end of the block is encountered (0–1).

| | |
|---|---|
| 0111<br>×0011 (0)<br>11111001 1–0<br>0000000 1–1<br>000111 0–1<br>00010101 (21)<br>(a) (7) × (3) = (21) | 0111<br>×1101 (0)<br>11111001 1–0<br>0000111 0–1<br>111001 1–0<br>11101011 (–21)<br>(b) (7) × (–3) = (–21) |
| 1001<br>×0011 (0)<br>00000111 1–0<br>0000000 1–1<br>111001 0–1<br>11101011 (–21)<br>(c) (–7) × (3) = (–21) | 1001<br>×1101 (0)<br>00000111 1–0<br>1111001 0–1<br>000111 1–0<br>00010101 (21)<br>(d) (–7) × (–3) = (21) |

**Figure 9.14** Examples Using Booth's Algorithm

## Operations involved in Booth's Algorithm:




| $Q_0$ $Q_{-1}$ | operation |
|---|---|
| 1 – 0 | (A) – (M) → (A); Then Right shift |
| 0 – 1 | (A) + (M) → (A); Then Right shift |
| 1 – 1 | only shift Right |
| 0 – 0 | only shift Right |

Examples:

1) SAL CX, 1 ; shift word in CX one bit position Left

2) MOV CL, 05H;
SAL AX, CL ; shift word in AX Left 5 times.

3) SHL DL, 01

4) SHR CX, 1

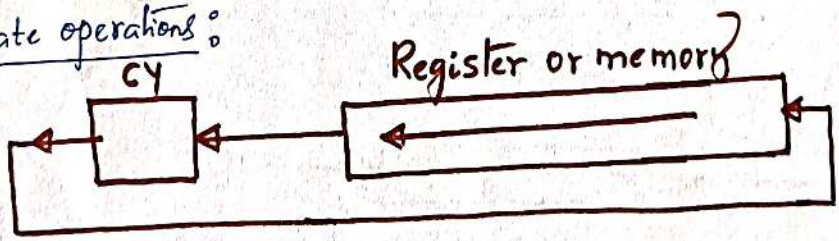
5) MOV CL, 05H
SHR AX, CL ; shift word in AX Right 5 times.

6) ~~SHR AX, CL~~

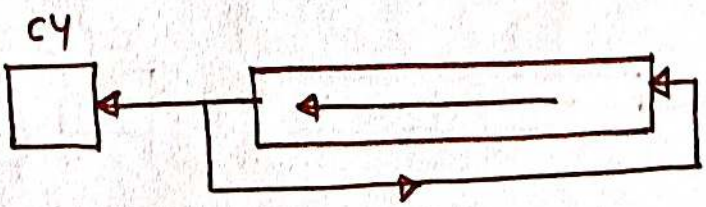
**Note:**
- Logical shift operations are used with unsigned Numbers.
- Arithmetic shift operations are used with Signed Numbers.

* Rotate Instructions:

Rotate operations:




RCL
(Rotate Left through carry)




ROL
(Rotate Left)




RCR
(Rotate Right through carry)




ROR
(Rotate Right)

## * Logic Instructions: (Bit Manipulation Instructions)

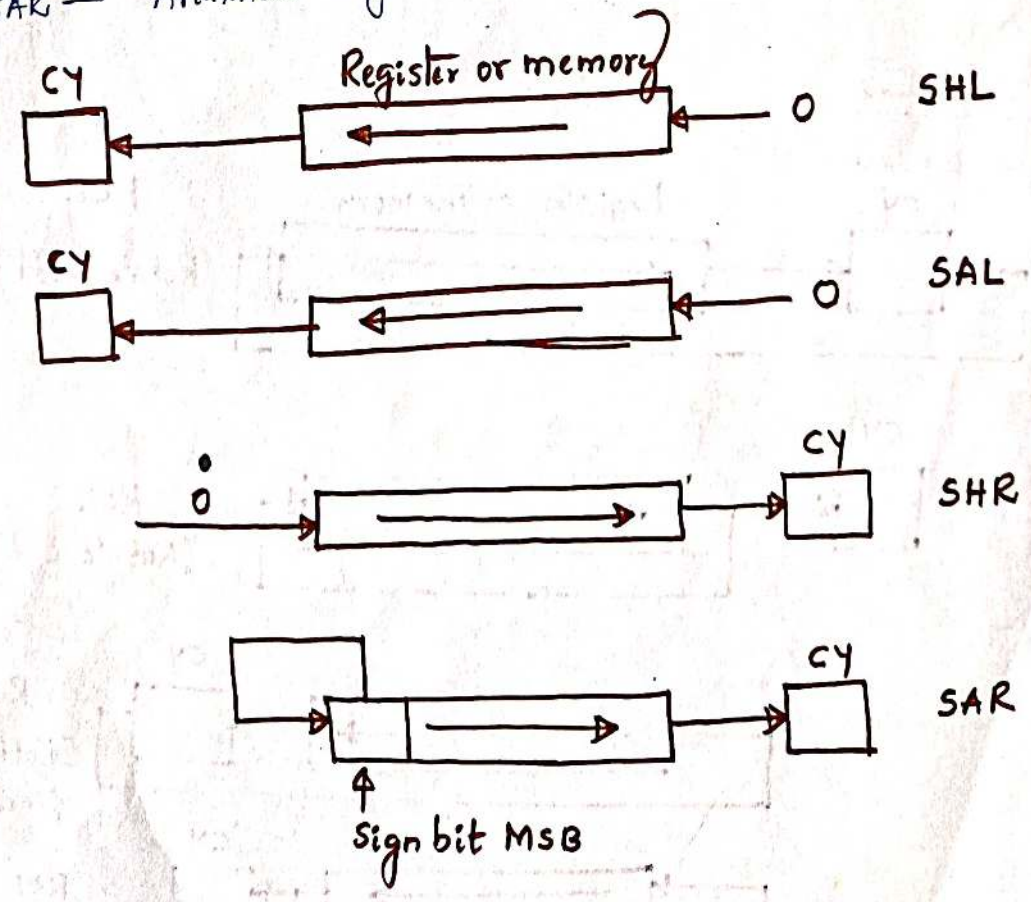
1) Logical Instructions
2) Shift Instructions
3) Rotate Instructions.

## * Shift Instructions:

- These instructions shift the operand word or byte bit by bit to the left or right.

Instructions:

SHL — Logical Left shift
SHR — Logical Right shift
SAL — Arithmetic Left shift
SAR — Arithmetic Right shift.



Examples:

1) ROL CX,1

2) MOL CL,03H
ROL BL,CL

3) MOV BL,7FH
ROL BL,01
HLT

4) ROR CX,1

5) MOV CL,03H
ROR BL,CL

6) RCL CX,1

7) MOV CL,04H
RCL AL,CL

8) RCR CX,1

9) MOV CL,04
RCR AL,CL

10) MOV BL,7FH
RCR BL,01H
HLT

Below, Compares typical one-, two- and three-address instructions for computing
$Y = (A-B)/(C+(D \times E))$

$Y = (A-B) \div (C+D \times E)$

Number of Addresses:

Ex:
```
SUB Y,A,B ;   Y ← A-B
MPY T,D,E ;   T ← D×E
ADD T,T,C ;   T ← T+C
DIV Y,Y,T ;   Y ← Y÷T
```

Three-Address instructions
Specifies two source operand Locations and a destination operand location.

NOTE: T → Temporary location. used to store some intermediate results.

Ex:
```
MOVE Y,A ;   Y ← A
SUB Y,B ;    Y ← Y-B
MOVE T,D ;   T ← D
MPY T,E ;    T ← T×E
ADD T,C ;    T ← T+C
DIV Y,T ;    Y ← Y÷T
```

Two-address Instructions

| | | |
|---|---|---|
| LOAD D; | AC ← D | AC: Accumulator |
| MPY E: | AC ← AC × E | |
| ADD C : | AC ← AC + C | |
| STOR Y : | Y ← AC | |
| LOAD A : | AC ← A | |
| SUB B ; | AC ← AC − B | |
| DIV Y; | AC ← AC ÷ Y | |
| STOR Y; | Y ← AC | |





## * Division:

### * operations:

~~a) shift A and Q in left direction by one bit.~~

1. Load the divisor into M Register and the Dividend into the A, Q registers.
2. The dividend must be expressed as a 2n-bit twos complement number

   Ex: The 4-bit 0111 becomes 00000111 and,
   The 4-bit 1001 becomes 11111001

3. Shift A, Q Left one bit position.
4. If M and A have the signs, perform (A) ← (A) − M; otherwise
   (A) ← (A)+(M)
5. The preceding operation is successful if the sign of A is the same before and after the operation.
   a) If the operation is successful or A=0, then Set $Q_0 \leftarrow 1$
   b) If the operation is unsuccessful and A≠0 then Set $Q_0 \leftarrow 0$
      and Restore the previous value of A.
6. Repeat Steps 3 through 5 as many times as there are bit positions in Q.
7. The Remainder is in A and the Quotient is in Q.

Example: 10 ⟶ Dividend ⟶ Q Register
3 ⟶ Divisor ⟶ M Register

| A | Q | M |
|---|---|---|

(P) $\frac{10}{2} = 5$

10 → Dividend → Q → Register

2 → Divisor → M → Register

$$\begin{array}{r} 5 = Q \\ 2\overline{)10} \\ 10 \\ \hline 0 = R \end{array}$$

| A | Q | M | |
|---|---|---|---|
| 0000 | 1010 | 0010 | |
| 0001 | 0100 | | Shift |
| | | | Subtract |
| 1111 | | | |
| | 0100 | | Restore |
| 0001 | | | Shift |
| 0010 | 1000 | | |
| | | | Subtract |
| 0000 | | | |
| | | | Set $Q_0 = 1$ |
| 0000 | 1001 | | Shift |
| 0001 | 0010 | | |
| | | | Subtract |
| 1111 | | | |
| | 0010 | | Restore |
| 0001 | | | |
| | 0100 | | Shift |
| 0010 | | | |
| | | | Add / Subtract |
| 0000 | | | |
| | 0101 | | Set $Q_0 = 1$ |
| 0000 | | | |
| Remainder | Quotient | | |

(A) = 0001

− (M) = 0010

↓ 1's

1101

\+ 1

1110

0001

1111

(A) = 0010

− (M) = 0010

↓ 1's

1101

\+ 1

1110

(A) = 0010

1 0000

(A) = 0001

− (M) = 0010

↓ 1's

1101

1

1110

0001

1111

(A) = 0010

(−) (+) (M) = 0010

1110

\+ 0010

1 0000

Example: $\frac{10}{3}$ → 10 → Dividend → Q Register
3 → Divisor → M Register

| A | Q | M | |
|---|---|---|---|
| 0000 | 1010 | 0011 | |
| | | | (A) = 0001 |
| | 0100 | Left shift | −(M) = 0011 |
| 0001 | | | 1100 |
| | | Subtract | + 1 |
| 1110 | | | 1101 |
| 0001 | 0100 | Restore | 0001 |
| | | | 1110 |
| | 1000 | shift | (A) = 0010 |
| 0010 | | Subtract | −(M) = 0011 |
| 1111 | | | 1100 |
| 0010 | 1000 | Restore | + 1 |
| | | | 1101 |
| | | Shift | 0010 |
| | 0000 | | 1111 |
| 0101 | | Subtract | (A) = 0101 |
| 0010 | | | (M) = 0011 |
| 0010 | 0001 | Set $Q_0$ = 1 | 1100 |
| | | Shift | + 1 |
| | 0010 | | 1101 |
| 0100 | | | 0101 |
| | | Subtract | 1 0010 |
| 0001 | | | |
| 0001 | 0011 | Set $Q_0$ = 1 | (A) = 0100 |
| | | | −(M) = 0011 |
| | | | 1100 |
| | | | + 1 |
| | | | 1101 |
| | | | 0100 |
| | | | 1 0001 |

Restore

Remainder

Quotient

$Y = A\bar{B} + B\bar{A}$
$= 01 + 10$
$= 1$

Example: 17 → Dividend → Q Register
3 → Divisor → M Register

| 16 | 8 | 4 | 2 | 1 |
|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 1 → 17 |
| 0 | 0 | 0 | 1 | 1 → 3 |

| A | Q | M | |
|---|---|---|---|
| 00000 | 10001 | 00011 | |
| 00001 | 00010 | Shift | (A) = 00001<br>- (M) = 00011 |
| 11110 | | Subtract | 11100<br>+ 1<br>11101 |
| 00001 | 00010 | Restore | 00001<br>(A) = 11110 |
| 00010 | 00100 | Shift | |
| 11111 | | Subtract | (A) = 00010 |
| 00010 | 00100 | Restore | - (M) = 00011<br>11101<br>00010<br>11111 |
| 00100 | 01000 | Shift | |
| 00001 | | Subtract | (A) = 00100 |
| 00001 | 01001 | Set $Q_0$=1 | - (M) = 00011<br>11101<br>00100<br>1 00001 |
| 00010 | 10010 | Shift | |
| 11111 | | Subtract | (A) = 00010 |
| 00010 | 10010 | Restore | - (M) = 00011<br>11101<br>(A) = 00010<br>(A) = 11111 |
| 00101 | 00100 | Shift | |
| 00010 | | Subtract | |
| 00010 | 00101 | Set $Q_0$=1 | (A) = 00101<br>- (M) = 00011<br>11101<br>00101<br>1 00010 |

Remainder — Quotient

## * Division Algorithm.

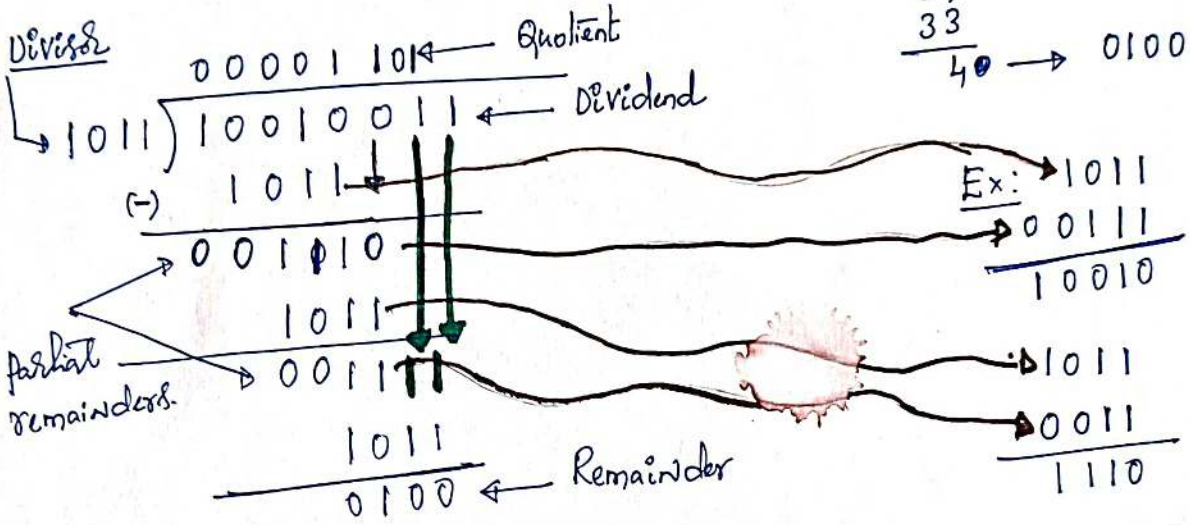
- Division is more complex than multiplication.

**Example:**

Dividend: 128 64 32 16 8 4 2 1
1 0 0 1 0 0 1 1 → 128+16+2+1 = 147

Divisor: 1011 → 8+2+1 = 11.

11) 147 (13 → 00001101
11
37
33
40 → 0100




**Operation**

* The bits of the dividend are examined from left to Right, until the sets of bits examined represents a number greater than or equal to the divisor. This is referred to as the divisor being able to the divide the number.
* until this event occurs, 0's are placed in the quotient from left to Right.
* when the event occurs, a 1 is placed in the quotient and the divisor is subtracted from the partial dividend.
* The process continues until all the bits of the dividend are exhausted

Algorithm:

Divisor ⟶ M Register

Dividend ⟶ Q Register

- At each step, A and Q Registers together are shifted to the left 1 bit.

## * Number of Addresses:

- One of the traditional ways of describing processor architecture is in terms of the number of addresses contained in each instruction.

- Ex: one address instructions, two-address instructions, three-address instructions

p) $Y = (A - B) \div (C + D \times E)$

1) One-Address Instructions:

```
LOAD D ;   AC ← D
MPY E ;    AC ← AC × E
ADD C ;    AC ← AC + C
STOR Y ;   Y ← AC
LOAD A ;   AC ← A
SUB B ;    AC ← AC - B
DIV Y ;    AC ← AC ÷ Y
STOR Y ;   Y ← AC
```

* A second address must be implicit.
* The accumulator contains one of the operands and is used to store the result.
* Eight instructions are needed to accomplish the task.

2) Two-address Instructions:

```
MOVE Y, A ;   Y ← A
SUB Y, B ;    Y ← Y - B
MOVE T, D ;   T ← D
MPY T, E ;    T ← T × E
ADD T, C ;    T ← T + C
DIV Y, T ;    Y ← Y ÷ T
```

* Six instructions.

3) Three-address instructions

```
SUB Y, A, B ;   Y ← A - B
MPY T, D, E ;   T ← D x E
ADD T, T, C ;   T ← T + C
DIV Y, Y, T ;   Y ← Y ÷ T
```

* Temporary Location, T, is used to store some intermediate values.